Bahasa Pemrograman Tingkat Rendah (MWCA010)

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

Format Instruksi Mesin

Pertemuan 5

Dosen Pengampu

Cecep Suwanda, S.Si., M.Kom.

Semester Ganjil

Capaian dan Tujuan Pembelajaran

Sub-CPMK 2.1

Menjelaskan format instruksi mesin (*opcode*, byte ModR/M, *displacement*/*immediate*) dan mengaitkannya dengan operasi yang dieksekusi perangkat keras.

CPMK yang didukung: CPMK02 ▪ **CPL yang didukung: CPL02**

Setelah pertemuan ini, mahasiswa mampu:

1. menjelaskan hubungan *mnemonic* assembly dan kode mesin yang dihasilkan assembler;
2. menjelaskan sebab satu *mnemonic* memiliki beberapa bentuk *encoding*;
3. menguraikan susunan field instruksi 8086 beserta urutannya;
4. memecah byte ModR/M menjadi field `mod`, `reg/opcode`, dan `r/m`;
5. melakukan *encoding* manual dan membacanya kembali dengan *disassembler*;
6. menilai pengaruh format instruksi terhadap ukuran kode dan kecepatan eksekusi.

Peta Materi

1. **Dari Mnemonic ke Kode Mesin**
   assembly sebagai simbol kode mesin; sebab satu mnemonic punya banyak *encoding*; format umum lima field; variasi panjang 1–6 byte; *little-endian*

2. **Byte ModR/M, Displacement, dan Immediate**
   field `mod`, `reg`/opcode, dan `r/m`; arti nilai `mod` dan `r/m`; dua pengecualian; peran *displacement* dan *immediate*; pengaruh terhadap ukuran dan kecepatan

3. **Encoding Manual dan Disassembler**
   lima langkah *encoding*; tabel *opcode* umum; perintah `U` pada `DEBUG`; keterbatasan *disassembly*

Praktik pada pertemuan ini

Menguraikan byte kode mesin lima instruksi secara manual, lalu membandingkannya dengan hasil perintah `U` dan menghitung selisih ukuran dua bentuk penulisan.

BAGIAN

Dari Mnemonic ke Kode Mesin

Assembly adalah Simbol dari Kode Mesin

- Perangkat keras hanya mengenal deretan bit. Instruksi memindahkan isi `BX` ke `AX` tidak pernah tersimpan sebagai huruf `MOV`; yang ada di memori adalah dua byte `89 D8h`.
- Kata pendek yang mewakili operasi tersebut disebut **mnemonic**. Assembler menerjemahkan setiap *mnemonic* beserta operandnya menjadi deretan byte kode mesin yang ekuivalen.
- `MOV` tidak selalu menghasilkan *opcode* yang sama: setiap bentuk operand memerlukan pengodean yang berbeda.

```
MOV AX, BX      <- yang ditulis programmer (mnemonic)
89 D8           <- yang tersimpan di memori (kode mesin)
```

Sumber: Bab VII, Assembly adalah Simbol dari Kode Mesin

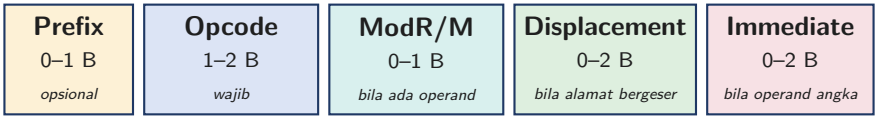
Mengapa Satu Mnemonic Memiliki Beberapa Encoding

Empat sebab utama:

1. **Jenis operand.** Register, sel memori, dan nilai segera adalah tiga jenis berbeda. `MOV AX, BX` memakai *opcode* `89h`, sedangkan `MOV AX, 1234h` memakai `B8h`.
2. **Ukuran operand.** `MOV AL, 5` dan `MOV AX, 5` beroperasi pada ukuran berbeda, sehingga *opcode*-nya `B0h` dan `B8h`.
3. **Arah perpindahan.** `8Bh` berarti "register ← memori", `89h` berarti "memori ← register".
4. **Mode pengalamatan.** Perbedaan alamat operand diatur lewat byte ModR/M dan *displacement*, bukan dengan mengganti *opcode*.

Akibatnya `MOV` memiliki puluhan bentuk mesin. Programmer tidak perlu menghafalnya, tetapi pembaca hasil *disassembly* harus mengenali bentuk yang dihadapi.

Format Umum Instruksi 8086



Urutan kelima field bersifat **tetap**; field yang tidak diperlukan tidak ditulis sama sekali, sehingga panjang instruksi bervariasi antara 1 sampai 6 byte.

Contoh pemakaian penuh: `MOV WORD PTR [BX+SI+1234h], 5678h` menghasilkan `C7 80 34 12 78 56h` — enam byte, semua field terpakai.

Sumber: Bab VII, Format Umum Instruksi 8086

Ukuran Instruksi Tidak Seragam

Instruksi	Kode mesin	Byte	Field yang dipakai
`NOP`	`90h`	1	Opcode
`PUSH AX`	`50h`	1	Opcode
`MOV AL, 5`	`B0 05h`	2	Opcode, immediate
`MOV AX, BX`	`89 D8h`	2	Opcode, ModR/M
`MOV AX, 1234h`	`B8 34 12h`	3	Opcode, immediate 16 bit
`ADD AX, [SI+4]`	`03 44 04h`	3	Opcode, ModR/M, displacement
`MOV WORD PTR [BX+SI+1234h], 5678h`	`C7 80 34h 12 78 56h`	6	Opcode, ModR/M, displacement, immediate

Intel membatasi panjang maksimum instruksi pada 6 byte; angka inilah yang menjadi alasan antrean pengambilan awal (*prefetch queue*) 8086 berukuran 6 byte.

Membaca Kode Mesin dari Kiri ke Kanan

Byte instruksi dieksekusi menurut urutan alamat, dari byte pertama sampai terakhir. Namun nilai 16 bit di dalamnya disimpan **little-endian**: byte bernilai rendah lebih dahulu.

```
MOV AX, 1234h     ; nilai immediate
```

Byte di memori

```
B8   34   12
```

byte `34h` lebih dahulu daripada `12h`

Kekeliruan yang paling sering terjadi

Membaca deretan `34 12h` sebagai `3412h` mengubah nilainya, bukan lagi `1234h`. Kesalahan urutan ini termasuk yang paling sering muncul pada analisis kode mesin.

Rangkuman Bagian 1

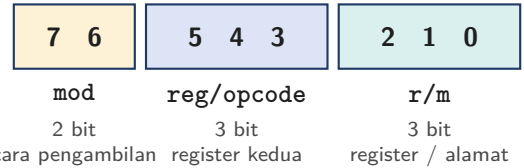
- *Mnemonic* assembly hanyalah simbol bagi kode mesin. Yang benar-benar dibaca prosesor adalah deretan byte.
- Satu *mnemonic* memiliki banyak bentuk *encoding* karena jenis operand, ukuran operand, arah perpindahan data, dan mode pengalamatan.
- Format instruksi 8086 tersusun dari lima field berurutan tetap: awalan, *opcode*, ModR/M, *displacement*, dan *immediate*.
- Field yang tidak diperlukan tidak ditulis, sehingga panjang instruksi berkisar 1 sampai 6 byte.
- Setiap nilai 16 bit disimpan *little-endian*: byte bernilai rendah lebih dahulu.

BAGIAN

Byte ModR/M, Displacement, dan Immediate

Struktur Byte ModR/M

Kehadiran byte ModR/M membuat satu *opcode* dapat melayani banyak kombinasi operand tanpa *opcode* baru untuk setiap kombinasi.



Contoh byte ModR/M `D8h` pada `MOV AX, BX` ditulis `11 011 000b`: `mod` = `11b`, `reg` = `011b` (`BX`), `r/m` = `000b` (`AX`). `mod` = `11b` berarti kedua operand berada di register.

Sumber: Bab VII, Struktur Byte ModR/M

Arti Nilai Field `mod`

Dua bit `mod` menentukan letak operand dan panjang *displacement* yang menyertainya.

`mod`	Biner	Displacement	Arti
0	`00b`	tidak ada	Operand memori; alamat langsung dari `r/m`
1	`01b`	1 byte (disp8)	Operand memori; alamat ditambah *displacement* 8 bit
2	`10b`	2 byte (disp16)	Operand memori; alamat ditambah *displacement* 16 bit
3	`11b`	tidak ada	Operand register; `r/m` menyatakan nomor register

Memakai `mod` = `01b` lebih hemat satu byte daripada `10b` untuk alamat yang sama, asalkan pergeserannya muat dalam 8 bit.

Arti Nilai Field r/m

Field r/m berperan ganda: penentu alamat efektif bila mod $\neq$ 11b, dan nomor register bila mod $=$ 11b.

Mode memori (mod $\neq$ 11b)

r/m	Biner	mod=00b	mod=01b/10b
0	000b	[BX+SI]	[BX+SI]+disp
1	001b	[BX+DI]	[BX+DI]+disp
2	010b	[BP+SI]	[BP+SI]+disp
3	011b	[BP+DI]	[BP+DI]+disp
4	100b	[SI]	[SI]+disp
5	101b	**langsung**	[DI]+disp
6	110b	[BP]	[BP]+disp
7	111b	[BX]	[BX]+disp

Mode register (mod $=$ 11b)

r/m	Reg 16	Reg 8
0	AX	AL
1	CX	CL
2	DX	DL
3	BX	BL
4	SP	AH
5	BP	CH
6	SI	DH
7	DI	BH

Sumber: Bab VII, Arti Nilai r/m

Dua Pengecualian dan Urutan Nomor Register

Dua pengecualian yang sering menjebak

1. Bila `mod` = `00b` dan `r/m` = `101b`, alamat efektifnya bukan `[DI]` melainkan **alamat langsung 16 bit**.
2. Bila `mod` $\neq$ `11b` dan `r/m` = `110b`, alamat efektifnya adalah `[BP]`.

Kedua aturan ini di luar pola umum dan harus dihafal.

Urutan nomor register 8086

`AX`, `CX`, `DX`, `BX`, `SP`, `BP`, `SI`, `DI` — mengikuti *encoding* asli Intel dan berlaku juga pada `PUSH` serta `POP`.

`mod` = `01b` dan `10b` memakai tabel alamat yang sama; hanya panjang *displacement*-nya berbeda. Contoh: `ADD AX, [SI+4]` memakai `mod` = `01b`, `r/m` = `100b`, ModR/M `44h`.

Peran Displacement dan Immediate

Kedua field terakhir sama-sama berupa bilangan, tetapi perannya berbeda.

Displacement Penambah pada alamat. Muncul hanya bila operand berupa memori dan `mod` bernilai `01b` atau `10b`. Pada `ADD AX, [SI+4]`, nilai `4h` adalah *displacement* sepanjang 8 atau 16 bit sesuai `mod`.

Immediate Data yang dioperasikan. Muncul bila operand berupa konstanta, misalnya `MOV AX, 1234h` atau `ADD BX, 10`. Panjangnya 8 atau 16 bit sesuai ukuran operand, dan nilainya selalu *little-endian*.

Keduanya dapat hadir bersamaan

`MOV WORD PTR [BX+SI+1234h], 5678h` memuat *displacement* dan *immediate* sekaligus, sehingga panjang totalnya enam byte. Urutan field tetap: *displacement* selalu mendahului *immediate*.

Pengaruh Format terhadap Ukuran dan Kecepatan

Bentuk operand	Panjang	Keterangan
Register ke register	2 byte	Paling ringkas; tanpa *displacement* dan tanpa *immediate*
Immediate 8 bit	2–3 byte	Nilai kecil; satu byte *immediate*
Immediate 16 bit	3–4 byte	Nilai besar; dua byte *immediate*
Memori `[SI]`	2–3 byte	`mod` = `00b`; tanpa *displacement*
Memori `[SI+4]`	3 byte	`mod` = `01b`; *displacement* 8 bit
Memori `[BX+SI+1234h]`	4 byte	`mod` = `10b`; *displacement* 16 bit

Semakin banyak field yang hadir, semakin panjang instruksi dan semakin banyak byte yang harus diambil melalui *prefetch queue*. Instruksi tiga byte umumnya selesai lebih cepat daripada instruksi enam byte.

Sumber: Bab VII, Pengaruh Format terhadap Ukuran dan Kecepatan

Rangkuman Bagian 2

- Byte ModR/M terbagi menjadi `mod` (2 bit), `reg/opcode` (3 bit), dan `r/m` (3 bit).
- Nilai `mod` menentukan letak operand dan panjang *displacement*: `00b` tanpa, `01b` disp8, `10b` disp16, dan `11b` operand register.
- `r/m` berperan ganda: penentu alamat efektif pada mode memori dan nomor register pada `mod` = `11b`.
- Dua pengecualian: `mod` = `00b` dengan `r/m` = `101b` berarti alamat langsung, sedangkan `r/m` = `110b` berarti `[BP]`.
- *Displacement* menggeser alamat, *immediate* menjadi operand. Bentuk operand menentukan ukuran kode dan jumlah byte yang diambil prosesor.

BAGIAN

Encoding Manual dan Disassembler

Lima Langkah Encoding Manual

Mengubah satu instruksi assembly menjadi byte kode mesin:

1. Tentukan operasi dan bentuk operandnya: register, memori, atau nilai segera — serta ukurannya (8 atau 16 bit).
2. Cari *opcode* dasar yang sesuai. Perhatikan arah perpindahan data, karena `8Bh` dan `89h` memiliki arti berlawanan.
3. Hitung byte ModR/M bila instruksi melibatkan register atau memori: susun field `mod`, `reg`, dan `r/m`, lalu gabungkan menjadi satu byte.
4. Tambahkan *displacement* bila operand memori bergeser, dan *immediate* bila operandnya berupa angka.
5. Tuliskan seluruh byte berurutan menurut alamat, dengan setiap nilai 16 bit dalam urutan *little-endian*.

Pekerjaan manual inilah yang dijalankan assembler untuk setiap baris kode sumber.

Sumber: Bab VII, Langkah-langkah Encoding Manual

Contoh Encoding Byte demi Byte

Instruksi	Byte	Rincian
`MOV AX, BX`	`89 D8h`	Opcode `89h` = "r/m16 ← r16". ModR/M `D8h` = `11 011 000b`: `mod` = `11b`, `reg` = `011b` (`BX`), `r/m` = `000b` (`AX`)
`MOV AL, 5`	`B0 05h`	Opcode `B0h` = "r8 ← imm8". Nomor `AL` = `0`, jadi `B0h` +`0` = `B0h`. Byte kedua `05h` adalah *immediate* 8 bit
`MOV AX, 1234h`	`B8 34 12h`	Opcode `B8h` = "r16 ← imm16". Nomor `AX` = `0`. *Immediate* `1234h` disimpan *little-endian*: `34h` lalu `12h`
`ADD AX, [SI+4]`	`03 44 04h`	Opcode `03h`. ModR/M `44h` = `01 000 100b`: `mod` = `01b` (disp8), `reg` = `000b` (`AX`), `r/m` = `100b` (`[SI]`). Byte ketiga `04h` adalah *displacement*
`INT 21h`	`CD 21h`	Opcode `CDh` = "panggil interupsi". Tidak memakai ModR/M; byte `21h` adalah *immediate* 8 bit

Tabel Ringkas Opcode: MOV dan Aritmatika

Instruksi	Opcode	Bentuk operand	Byte
`MOV`	`88h–8Bh`	Antar `r/m` dan register	2–4
`MOV`	`B0h–B7h`	Register 8 bit, *immediate* 8 bit	2
`MOV`	`B8h–BFh`	Register 16 bit, *immediate* 16 bit	3
`MOV`	`C6h/C7h`	`r/m`, *immediate*	3–6
`ADD`	`00h–03h`	Antar `r/m` dan register	2–4
`ADD`	`04h/05h`	`AL/AX` dengan *immediate*	2–3
`SUB`	`28h–2Dh`	Pola sama dengan `ADD`	2–4
`CMP`	`38h–3Dh`	Pembandingan; hanya mengubah *flag*	2–4

`00h–03h` berarti empat *opcode* berurutan: `00h` untuk 8 bit arah `r/m` ← `reg`, `01h` untuk 16 bit, lalu `02h` dan `03h` untuk arah sebaliknya.

Tabel Ringkas Opcode: Lompatan, Interupsi, dan Stack

Instruksi	Opcode	Bentuk operand	Byte
`JMP`	`EBh`	Lompat pendek, *rel8*	2
`JMP`	`E9h`	Lompat dekat, *rel16*	3
`CALL`	`E8h`	Panggil dekat, *rel16*	3
`INT`	`CCh`	`INT 3`; titik henti *debugger*	1
`INT`	`CDh`	Interupsi dengan nomor 8 bit	2
`PUSH`	`50h`–`57h`	Dorong register 16 bit ke *stack*	1
`POP`	`58h`–`5Fh`	Ambil register 16 bit dari *stack*	1

Pola kelompok

Menghafal seluruh *opcode* 8086 tidak diperlukan; mengenali pola kelompoknya jauh lebih berguna.

Sumber: Bab VII, Tabel Ringkas Opcode Instruksi Umum

Membalik Kode Mesin dengan Disassembler

Disassembler melakukan pekerjaan kebalikan assembler: membaca byte kode mesin dan menampilkannya kembali sebagai *mnemonic*. `DEBUG` menyediakan perintah `U` (*unassemble*) untuk keperluan ini.

```
-u 100 10C
13F2:0100 B80500    MOV   AX,0005
13F2:0103 BB0100    MOV   BX,0001
13F2:0106 8908      MOV   [BX+SI],CX
13F2:0108 03C3      ADD   AX,BX
13F2:010A CD21      INT   21
13F2:010C 90        NOP
```

Setiap baris memuat tiga kolom: alamat, byte kode mesin, lalu *mnemonic*. Baris pertama menempati tiga byte, sedangkan `ADD AX, BX` hanya dua byte.

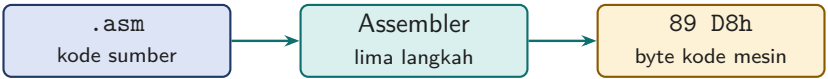
Keterbatasan yang harus diingat

Disassembler tidak dapat membedakan kode dari data. Hasilnya harus selalu diperiksa terhadap titik masuk dan alur kendali program.

Hubungan dengan Translasi Program

Encoding manual adalah inti proses translasi

- Assembler menjalankan lima langkah *encoding* yang sama untuk setiap baris kode sumber, tetapi jauh lebih cepat dan teliti.
- Direktif, tabel simbol, dan penanganan label dibahas pada Bab IX.
- Dengan memahami format instruksi lebih dahulu, setiap keputusan assembler dapat ditelusuri sampai byte demi byte.



Rangkuman Bagian 3

- *Encoding* manual ditempuh lewat lima langkah: tentukan operasi dan operand, cari *opcode*, susun ModR/M, tambahkan *displacement*/*immediate*, lalu tulis *little-endian*.
- Sebagian *opcode* memakai tiga bit `reg` sebagai penambah, seperti `B0h` dan `B8h` yang nomor register-nya disisipkan pada bit terendah.
- Instruksi yang tidak melibatkan register atau memori sama sekali tidak memerlukan byte ModR/M, misalnya `INT 21h`.
- Perintah `U` pada `DEBUG` mencetak alamat, byte kode mesin, dan *mnemonic* hasil terjemahan.
- Disassembler tidak membedakan kode dari data; hasilnya harus diperiksa terhadap titik masuk dan alur kendali program.

Contoh 1: Kode Sumber `opcode-demo.asm`

Lima instruksi dengan bentuk operand berbeda, siap dibongkar dengan perintah `U`.

```
.MODEL SMALL
.DATA
    pesan DB 'Format instruksi mesin', 0Dh, 0Ah, '$'
.CODE
START:
    MOV AX, @DATA
    MOV DS, AX
    MOV AX, 0005h       ; B8 05 00
    MOV BX, 0001h       ; BB 01 00
    ADD AX, BX          ; 03 C3
    MOV CX, AX          ; 89 C1
    SUB AX, 0002h       ; 2D 02 00
    ...                 ; cetak pesan dan keluar dari program
END START
```

Bagian pencetakan pesan dan keluar program (`MOV AH, 09h`, `LEA DX, pesan`, `INT 21h`, `MOV AH, 4Ch`) dihilangkan agar ringkas; berkas lengkapnya ada pada `code/asm/opcode-demo.asm`.

Contoh 1: Hasil Bongkar dan Perbandingannya

```
-u
0B2A:0005 B80500    MOV   AX,0005
0B2A:0008 BB0100    MOV   BX,0001
0B2A:000B 03C3      ADD   AX,BX
0B2A:000D 89C1      MOV   CX,AX
0B2A:000F 2D0200    SUB   AX,0002
```

Kode sumber	Kode mesin	Byte	Field yang tampak
`MOV AX, 0005h`	`B8 05 00h`	3	Opcode penambah register, imm. 16 bit
`MOV BX, 0001h`	`BB 01 00h`	3	Nomor `BX` = 3, jadi `B8h` +3
`ADD AX, BX`	`03 C3h`	2	ModR/M `11 000 011b`: `reg` = `AX`, `r/m` = `BX`
`MOV CX, AX`	`89 C1h`	2	Opcode `89h` arah `r/m` ← `reg`
`SUB AX, 0002h`	`2D 02 00h`	3	Opcode AX dengan immediate 16 bit

Contoh 2: Kode Sumber `ukuran-instruksi.asm`

Dua penulisan yang menghasilkan keadaan register sama, tetapi menempati jumlah byte berbeda.

```
1  .MODEL TINY
2  .CODE
3  ORG 100h
4  START:
5      MOV AX, 0001h       ; bentuk A, 3 byte
6      MOV BX, 0001h       ; bentuk A, 3 byte
7      MOV BX, 0001h       ; bentuk B, 3 byte
8      MOV AX, BX          ; bentuk B, 2 byte
9      ...
10 pesan DB 'Dua bentuk, hasil sama, ukuran berbeda.', 0Dh, 0Ah, '$'
11 END START
```

Program berbentuk `.COM`, sehingga seluruh instruksi dimulai pada offset `100h` dan `DS` tidak perlu diinisialisasi.

Contoh 2: Hasil Bongkar dan Jumlah Byte

```
-u 100 10A
13F2:0100 B80100    MOV   AX,0001   ; bentuk A, 3 byte
13F2:0103 BB0100    MOV   BX,0001   ; bentuk A, 3 byte
13F2:0106 BB0100    MOV   BX,0001   ; bentuk B, 3 byte
13F2:0109 89D8      MOV   AX,BX     ; bentuk B, 2 byte
```

Bentuk	Instruksi	Kode mesin	Byte
A	`MOV AX, 0001h`	`B8 01 00h`	3
A	`MOV BX, 0001h`	`BB 01 00h`	3
Jumlah bentuk A			**6**
B	`MOV BX, 0001h`	`BB 01 00h`	3
B	`MOV AX, BX`	`89 D8h`	2
Jumlah bentuk B			**5**

Selisih satu byte berasal dari *immediate* 16 bit kedua yang tidak lagi diperlukan. Bentuk B belum tentu lebih baik: ia bergantung pada nilai register lain dan lebih sulit dibaca.

Praktikum: Analisis Opcode Manual dan Disassembler

Judul praktikum

Analisis *opcode* manual dan *disassembler* ▪ **Sub-CPMK:** 2.1 ▪ **Waktu:** 100 menit ▪ **Bentuk:** individu ▪ **Tugas 2**

Alat dan bahan

- Komputer bersistem operasi Windows
- Emu8086, GUI Turbo Assembler (TASM/TLINK), atau DOSBox dengan `DEBUG`
- Berkas `opcode-demo.asm`, `ukuran-instruksi.asm`, dan lembar kerja *encoding*

Langkah kerja

1. Isi lembar kerja *encoding* lima instruksi secara manual
2. Rakit `opcode-demo.asm`; jalankan `U` pada `DEBUG`
3. Cocokkan tiap baris hasil bongkar dengan kode sumber
4. Rakit `ukuran-instruksi.asm`; hitung byte bentuk A dan B
5. Ubah `MOV AX, 1234h` menjadi `MOV AX, 3412h`; jelaskan *little-endian*-nya

Dikumpulkan: lembar kerja *encoding* terisi, keluaran perintah `U`, perhitungan byte bentuk A dan B, jawaban penelusuran *immediate*, dan jurnal praktikum.

Latihan

1. Jelaskan hubungan antara *mnemonic* dan *opcode*. Mengapa programmer cukup menulis `MOV AX, BX` tanpa menentukan sendiri byte `89 D8h`?
2. Instruksi berbyte `C7 80 34 12 78 56h`. Tentukan *opcode* dan operasinya, nilai biner `mod`, `reg`, dan `r/m`, serta panjang dan isi *displacement* dan *immediate*.
3. Tuliskan byte kode mesin untuk `MOV DX, 00FFh`, `ADD AL, 3`, dan `MOV [BX], AL` beserta langkah perhitungannya.
4. Jelaskan perbedaan peran *displacement* dan *immediate*. Kapan keduanya muncul, dan dapatkah keduanya muncul sekaligus?
5. Dua program berakhir sama. Program pertama memakai `MOV AX, 0001h` dan `MOV BX, 0001h`; program kedua memakai `MOV BX, 0001h` dan `MOV AX, BX`. Hitung jumlah byte keduanya dan sebutkan field penyebab selisihnya.

Rangkuman

- *Mnemonic* assembly hanyalah simbol bagi kode mesin. Satu mnemonic memiliki banyak bentuk *encoding* karena jenis operand, ukuran operand, arah perpindahan data, dan mode pengalamatan.
- Format instruksi 8086 tersusun dari lima field berurutan tetap: awalan, *opcode*, ModR/M, *displacement*, dan *immediate*. Panjang totalnya 1 sampai 6 byte.
- Byte ModR/M terbagi menjadi `mod` (2 bit), `reg/opcode` (3 bit), dan `r/m` (3 bit). Nilai `mod` menentukan letak operand dan panjang *displacement*.
- *Displacement* menggeser alamat efektif dan hanya muncul pada operand memori; *immediate* adalah konstanta yang menjadi operand. Keduanya dapat muncul bersamaan.
- Setiap nilai 16 bit disimpan *little-endian*, dan bentuk operand mempengaruhi ukuran kode serta jumlah byte yang diambil prosesor.
- Disassembler, misalnya perintah `U` pada `DEBUG`, membalik kode mesin menjadi *mnemonic*; hasilnya harus diperiksa karena disassembler tidak membedakan kode dari data.

Cek Pemahaman

1. Urutan field instruksi 8086 dari kiri ke kanan adalah ...
   1.1 *opcode*, awalan, ModR/M, *immediate*, *displacement*
   1.2 awalan, *opcode*, ModR/M, *displacement*, *immediate*
   1.3 ModR/M, *opcode*, awalan, *displacement*, *immediate*
   1.4 awalan, ModR/M, *opcode*, *immediate*, *displacement*

2. Byte ModR/M `D8h` memiliki nilai `mod` sebesar ...
   2.1 `00b`, operand memori tanpa *displacement*
   2.2 `01b`, operand memori dengan disp8
   2.3 `10b`, operand memori dengan disp16
   2.4 `11b`, operand berupa register

3. `MOV AX, 1234h` dikodekan `B8 34 12h` dan bukan `B8 12 34h` karena ...
   3.1 8086 menyimpan nilai 16 bit secara *little-endian*
   3.2 *opcode* `B8h` hanya melayani nilai kecil
   3.3 urutan byte ditentukan oleh mode pengalamatan
   3.4 byte rendah disimpan pada `AH`

Penutup

Sebelum pertemuan berikutnya

- Selesaikan Tugas 2: lembar kerja *encoding* terisi, keluaran perintah `U`, dan perhitungan byte dua bentuk penulisan.
- Latih kembali pemecahan byte ModR/M dan urutan *little-endian* sebelum mempelajari mode pengalamatan.
- Baca **Bab VIII** — Mode Pengalamatan.

Terima kasih

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil